CLOUD PLATFORM PORTFOLIO

JO

YOUNG

DAE

Cloud Infrastructure Engineer / Platform Engineer

운영을 고려한 설계로 안정적인 플랫폼을 만드는 엔지니어

ABOUT ME

운영을 먼저 생각하는 엔지니어

문제를 임시로 해결하는 것이 아니라 원인을 끝까지 분석하여 재발하지 않는 구조를 만드는 것을 중요하게 생각합니다. Terraform 기반 AWS 인프라 구축 프로젝트와 KVM + libvirt 기반 On-Premise Kubernetes Platform 프로젝트를 수행하며 운영 중심 설계와 플랫폼 자동화 경험을 쌓았습니다.

Operation First
구축 이후의 운영과 장애 대응을 먼저 고려합니다.

Problem Solving
문제를 임시로 넘기지 않고 원인을 끝까지 추적합니다.

GitOps
반복 작업을 줄이고 운영 효율을 높입니다.

Collaboration
운영 시나리오와 근거를 바탕으로 팀과 의사결정합니다.

SKILLS

Technical Skills

Cloud

AWS Azure

IaC

Terraform

Container

Docker Kubernetes

Virtualization

KVM libvirt containerd Cilium

AI

Azure OpenAI

Programming

Python Node.js TypeScript

Version Control

Git GitHub

EDUCATION

KT Cloud Infra Bootcamp

2025.12 ~ 2026.07

클라우드 인프라 전문 과정에서 AWS, Terraform, Docker, Kubernetes, CI/CD, Private Cloud, Azure 기반 실습과 프로젝트를 수행했습니다.

- AWS
- Terraform
- Docker
- Kubernetes
- CI/CD
- Private Cloud
- Azure

PROJECT SUMMARY

Projects

PROJECT 01

Terraform 기반 AWS Infrastructure

Cloud Infrastructure Engineer

2026.01.26 ~ 2026.03.05

Contribution 50%

PROJECT 02

Kubernetes Internal Developer Platform

AIOps Engineer

2026.03.10 ~ 2026.04.10

Contribution 60%

PROJECT 03

AWS기반 AIOps+FinOps Platform

Cloud Infrastructure Engineer

2026.04.16 ~ 2026.07.21

Contribution 50%

PROJECT 01

Terraform 기반 AWS Infrastructure

Overview

- Terraform 기반으로 AWS 인프라를 코드화하여 반복 가능한 구축 환경을 구현했습니다.
- 운영 환경을 고려한 네트워크 설계와 Auto Scaling 구성을 통해 안정적인 인프라 구축을 목표로 했습니다.



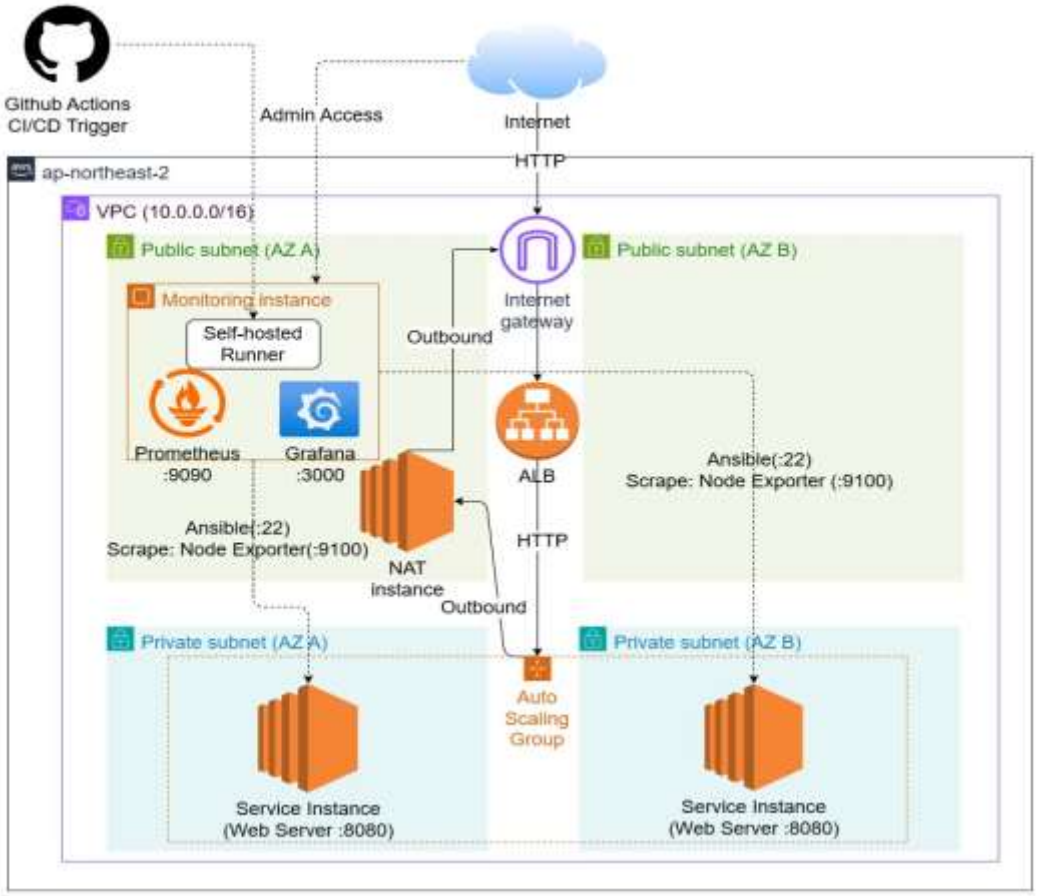
Demo QR

Background

- 운영 환경에서는 동일한 인프라를 반복적으로 구축해야 하는 상황이 자주 발생합니다.
- 수작업 구성은 환경 차이와 설정 오류로 이어질 수 있어 Infrastructure as Code를 적용했습니다.

PROJECT 01 ARCHITECTURE

AWS Architecture



MY CONTRIBUTION

Infrastructure Components

Terraform | ALB | ASG | EC2 | NAT Instance

Ubuntu | Nginx | Git | PR

기여 요약

- Terraform 기반 AWS 인프라 구축
- VPC, EC2, ALB, Auto Scaling 구성
- 운영 안정성을 고려한 NAT Instance 역할 분리 제안

TECHNICAL DECISIONS

왜 이렇게 설계했는가

Terraform
Infrastructure as Code 적용으로 반복 가능한 인프라 구성

ALB
가용성을 고려한 트래픽 분산

ASG
확장 가능한 서비스 인스턴스 구성

NAT Instance
운영성과 장애 분석을 고려한 역할 분리

TROUBLESHOOTING

운영 안정성을 위한 역할 분리

Problem

비용 절감을 위해 NAT Instance와 Private Cloud Instance를 통합하자는 의견이 있었습니다.

Risk

장애 발생 시 역할이 혼재되면 원인 분석과 복구가 어려워질 수 있다고 판단했습니다.

Solution

운영 시나리오를 기준으로 역할 분리의 필요성을 설명하고 팀 논의를 통해 구조를 수정했습니다.

Result

운영 안정성과 장애 대응성을 고려한 인프라 구조를 설계할 수 있었습니다.

LESSONS LEARNED

IaC는 자동화보다 운영 표준화에 가깝다

Terraform을 통해 인프라를 코드로 관리하면서, IaC는 단순히 리소스를 빠르게 만드는 기술이 아니라 동일한 운영 환경을 반복 가능하게 만드는 과정이라는 점을 배웠습니다.

PROJECT 02

Kubernetes Internal Developer Platform

Overview

- 실제 기업 환경을 가정해 개발자에게 편리한 개발 플랫폼을 제공하고, 운영자에게는 반복적인 운영 업무를 줄일 수 있는 IDP를 구축했습니다.
- Discord ChatOps, Backstage AI Assistant, Azure OpenAI를 활용해 Kubernetes 운영 정보를 자연어로 조회할 수 있도록 설계했습니다.

Background

- Kubernetes와 MSA 환경이 확대되며 개발자는 여러 운영 도구를 직접 사용해야 했고, 운영자는 반복적인 문의와 장애 대응 업무를 수행해야 했습니다.
- 이러한 운영 부담을 줄이기 위해 자연어 기반 운영 지원 플랫폼을 구현했습니다.

PROJECT 02 INFRASTRUCTURE

KVM 기반 Virtual Machine 4대 구성

단일 고사양 서버를 KVM + libvirt 기반으로 가상화하여 Control Plane 1대와 Worker Node 3대로 분리하고, Kubernetes Cluster 구축을 위한 On-Premise 환경을 구성했습니다.

Virtualization Structure

Host Server
Ubuntu 24.04 · 32 Core · 128GB RAM

KVM + libvirt
Linux Native Hypervisor

k8s-cp · Control Plane

k8s-w1 · Worker 1

k8s-w2 · Worker 2

k8s-w3 · Worker 3

VM Resource Planning

VM	Role	CPU	RAM	Disk
k8s-cp	Control Plane	4	16GB	100GB
k8s-w1	Worker 1	8	32GB	400GB
k8s-w2	Worker 2	8	32GB	400GB
k8s-w3	Worker 3	8	32GB	400GB

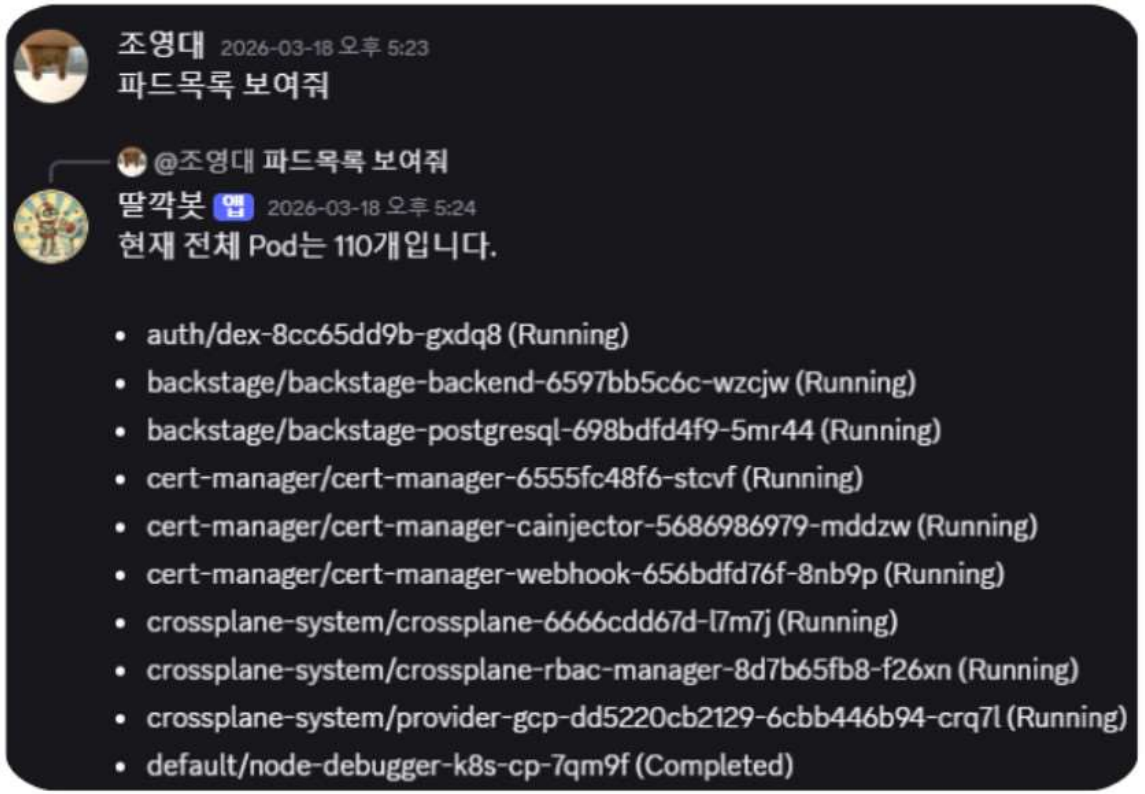
On-Premise Platform

PROJECT 02 ARCHITECTURE



CORE FEATURE

Discord ChatOps

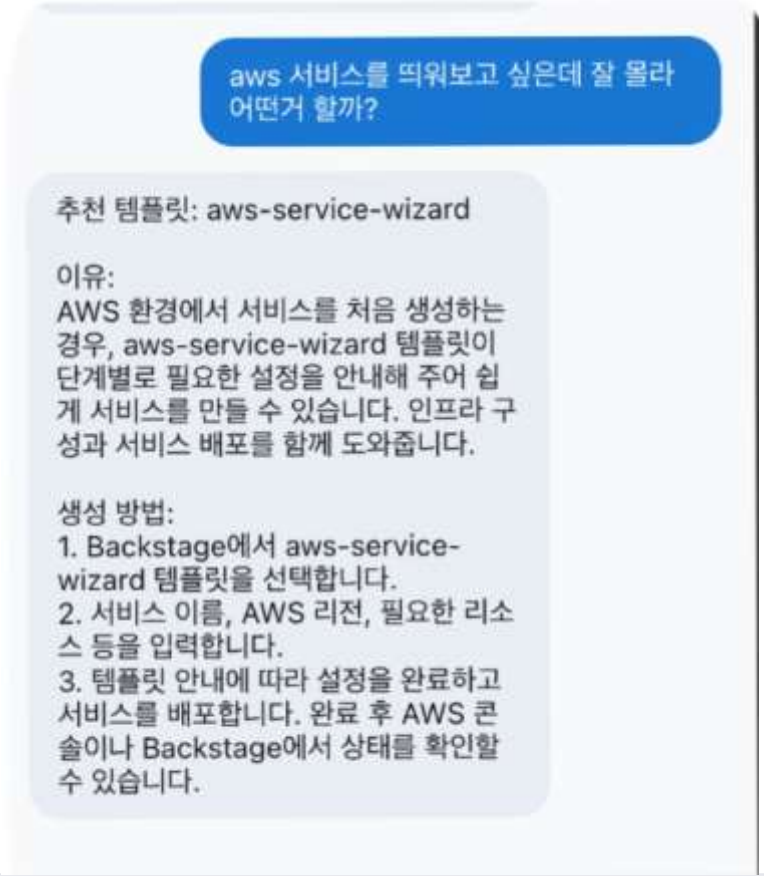


기능

- Pod 상태 조회
- 로그 조회 장
- 애 분석
- Service 조회

CORE FEATURE

Backstage AI Assistant



역할

- 포털 기반 자연어 질의
- Kubernetes 운영 지원 서비
- 스 생성 템플릿 추천 개발자
- 셀프서비스 경험 제공

GITOPS & DECISIONS

ArgoCD Pipeline



Technical Decisions

- Discord: 운영자가 익숙한 협업 도구를 활용해 접근성 향상
- Backstage: Internal Developer Platform 역할 수행
- Azure OpenAI: 자연어 기반 운영 질의와 장애 분석 지원
- Kubernetes API: 실시간 클러스터 운영 정보 제공
- Fallback: previous → current → describe 순서로 분석 가능성 확보

TROUBLESHOOTING

운영 중단을 막기 위한 예외 처리 구조

Kubernetes API
API 오류가 Bot 중단으로 이어지는 문제를 try-catch와 fallback 처리로 개선했습니다.

Azure OpenAI
Proxy, API Key, Deployment Name, CORS 문제를 순차적으로 분석하고 해결했습니다.

Natural Language
잘못 분류되는 질문을 intent 우선순위 조정과 토큰화 방식으로 개선했습니다.

Fallback
로그가 없을 경우 previous → current → describe 순서로 분석 가능성을 확보했습니다.

PROJECT 03

PROJECT 03
AWS기반 AIOps+FinOps Platform

Overview

- AWS 운영 데이터를 수집·분석하여 이상 탐지와 비용 분석을 수행하는 AIOps·FinOps 통합 플랫폼을 구축했습니다.
- 서울 운영 리전과 오사카 DR 리전을 구성하고 Human Gate 기반 Semi-Automated DR을 설계했습니다.

Background

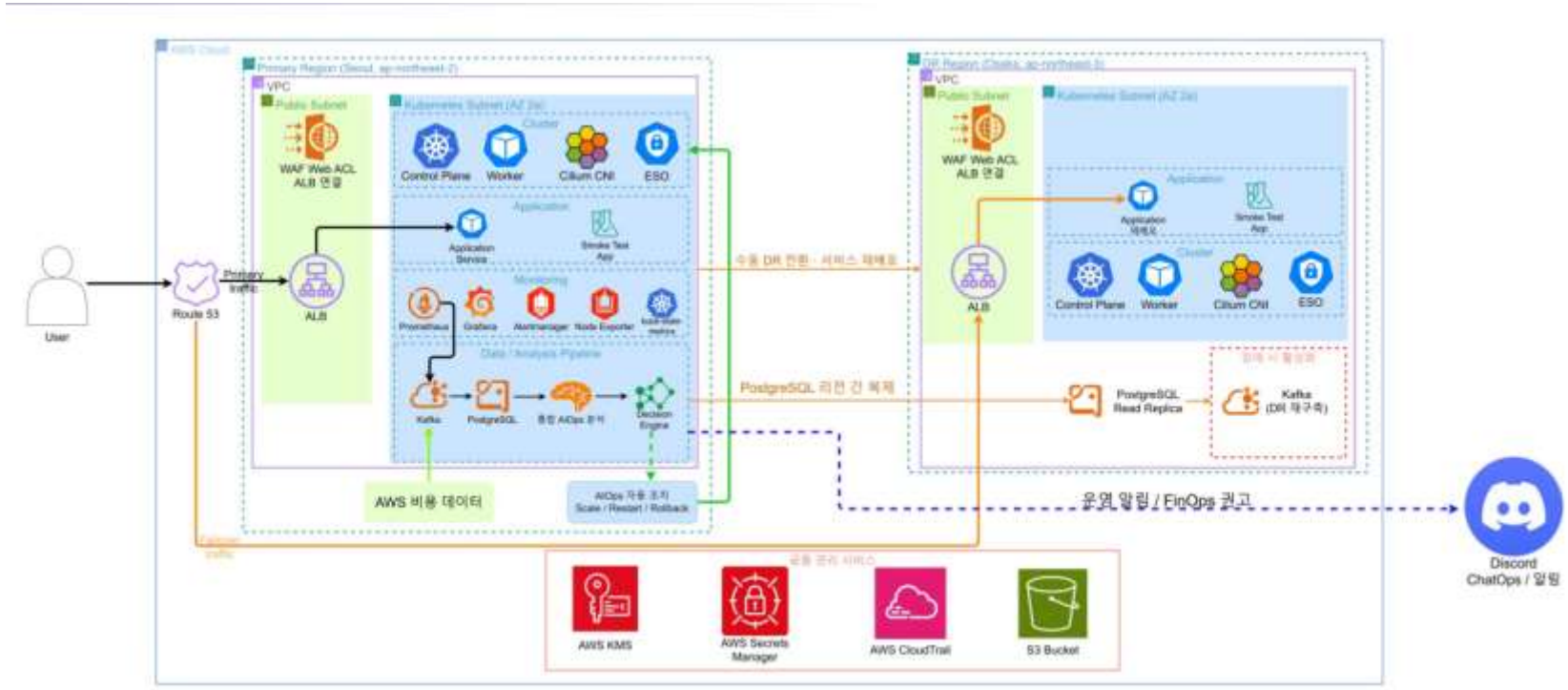
- 클라우드 장애가 여러 서비스로 확산될 수 있어 장애를 신속하게 탐지하고 대응할 수 있는 운영 체계가 필요했습니다.
- 분산된 운영·비용 데이터를 통합하고 AIOps·FinOps 분석과 DR을 하나의 운영 흐름으로 연결하고자 했습니다.



AIOps+FinOps

PROJECT 03 ARCHITECTURE

AWS Architecture



CORE FEATURE

AIOps + FinOps Analysis

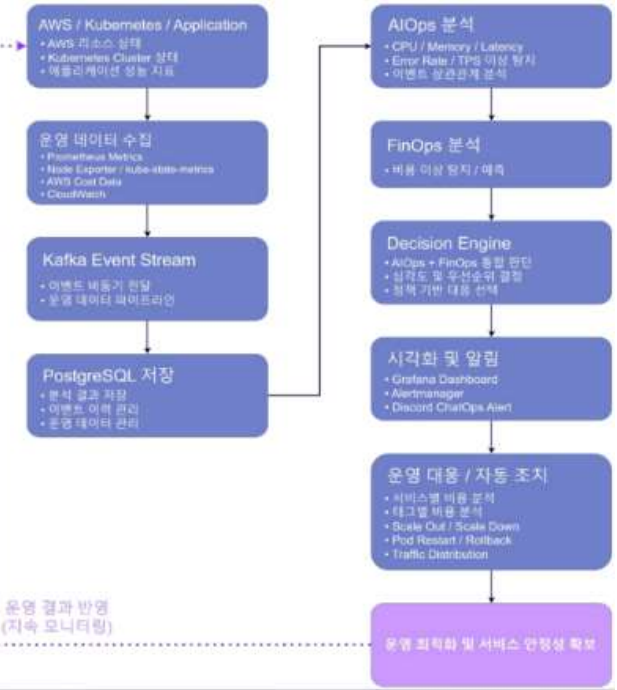
Operations Workflow

1.운영 데이터 통합 수집
- Prometheus · CloudWatch 기반 운영 지표 수집
- AWS Cost Data 연계해 운영 상태와 비용 정보를 함께 확보

2. 이벤트 기반 데이터 파이프라인
- 수집 데이터를 Kafka 로 비동기 전달해 모듈 간 결합도를 낮춤
- PostgreSQL 에 운영 이벤트 , 분석 결과와 이력을 저장

3. AIOps · FinOps 통합 분석
- AIOps 는 성능 이상과 장애 징후를 , FinOps 는 비용 이상과 변화 추세를 분석
- Decision Engine 에서 심각도와 대응 정책 결정

4. 시각화 · 알림 및 자동 조치
- Grafana 로 결과를 시각화하고 Alertmanager · Discord 로 운영 알림 전달
- Scale Out/Down, Pod Restart, Rollback, Traffic Distribution 을 정책에 따라 수행
- 조치 결과를 다시 수집해 지속 모니터링과 정책 재평가

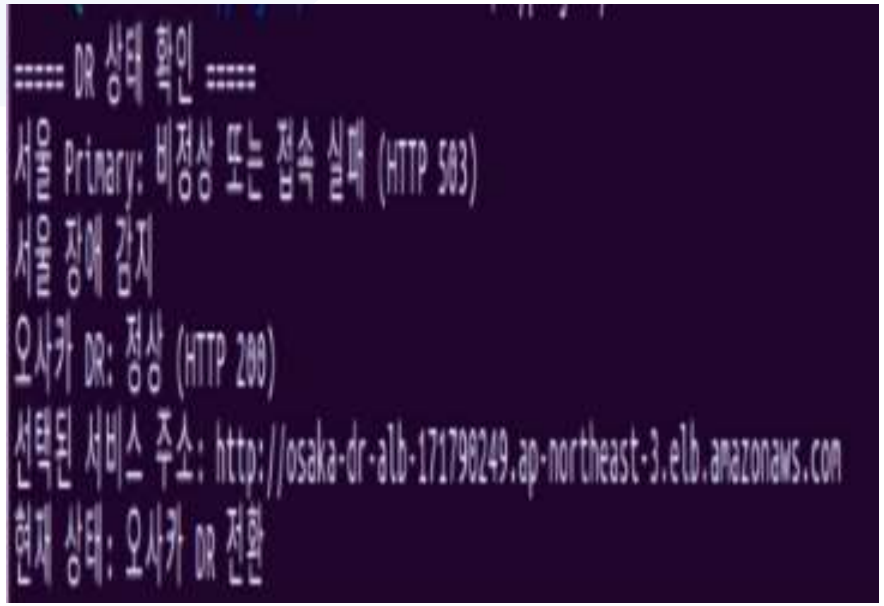


기능

- Prometheus 기반 운영 메트릭 수집
- AIOps 기반 이상 징후 탐지
- FinOps 기반 비용분석 및 예측
- Decison Engine 기반 대응 판단

CORE FEATURE

Semi-Automated DR



기능

- AWS Seoul → Osaka 멀티 리전 DR
- RDS Read Replica 기반 데이터 복제
- Route53 Failover를 통한 트래픽 전환
- Smoke Test를 통한 서비스 복구 검증

TROUBLESHOOTING

인프라 운영 안정화를 위한 문제 해결

RDS Migration

초기 RDS의 VPC 구성이 Kubernetes 환경과 일치하지 않아 연결 문제가 발생했습니다. 동일 VPC 기준으로 RDS를 재구성하고 Security Group과 5432 통신을 점검하여 연결을 정상화했습니다.

Kafka Network

Kafka Broker가 공인 IP를 사용하던 구조를 Worker-2의 Private IP 기반 통신으로 변경하고 Data Collector의 Broker 주소를 수정하여 내부 통신 구조로 개선했습니다..

Kubernetes Storage

Kubernetes 노드의 Root EBS 용량 부족으로 DiskPressure가 발생하여 EBS를 8GB에서 20GB로 확장하고 파일시스템을 반영하여 노드를 정상 상태로 복구했습니다.

Disaster Recovery

서울 운영 환경과 오사카 DR 환경을 분리하고 RDS 복제 및 Route53 Failover 흐름을 구성했습니다. 오탐에 의한 불필요한 전환을 줄이기 위해 HumanGate 기반 승인 절차를 적용했습니다.

CONTACT

안정적인 플랫폼을 함께 만들어가겠습니다.

GitHub: https://github.com/YoungDae-Jo
Email: qsxwdv961@gmail.com
Portfolio: youngdae-cloud-portfolio.vercel.app

Terraform Demo

IDP Demo

AIOps+FinOpos